STERLING · SBG FUNDING AI GUIDE

Project walkthrough

docs/PROJECT_WALKTHROUGH.md · October 29, 2026

A plain-language explanation of the whole project, for explaining it confidently in interviews. It covers what Sterling does, the tech stack and why each piece is there, the architecture, how every main feature flows through the code, the ten most important functions, resume points, 20 likely interview questions with answers, and what to add next.

Everything here was checked against the code in October 2026: all 100 tests pass, and the search-tuning numbers come from a fresh run. The project has no database and no login, and the answers say so plainly where a question assumes them. The last section lists what hasn't been verified yet.

Contents

1. Project overview
2. Tech stack
3. Architecture
4. Code flow
5. Important functions
6. Resume points
7. Interview questions
8. Improvements that would make it stand out
9. Not verified yet

---

1. Project overview

In one line: Sterling is a chatbot for SBG Funding's website. It helps a small-business owner find the right financing, see whether they meet SBG's minimums, estimate how much they could get, and reach a human when that's the better path.

The problem it solves

- On sbgfunding.com the answers are spread across product pages, an FAQ, two calculators, an application and a phone line. The owner has to piece it together alone, and only finds out whether they qualify after applying or calling.
- It costs SBG too. Advisors spend time on people who don't meet the minimums. Customers who get passed to a person have to repeat their story.

Main features

1. **Guided questions.** It asks what an SBG advisor would ask: what the money is for, how much, how soon, business type (LLC / Partnership / Corporation / Sole Proprietor), industry, time in business, revenue and credit. Users click answer cards, or type one sentence ("roofing LLC, 3 years, $80K a month") and several fields fill at once.
2. **Eligibility check.** It compares the business with SBG's four published minimums: 500+ credit score, 6+ months in business, $150K+ a year in revenue, and 4 bank statements. It then ranks SBG's 9 financing products as "strong fit", "worth discussing" or "not a fit yet", with reasons and what would change the result.
3. **Funding calculator.** It uses the same formulas as SBG's public online calculators to estimate the maximum amount, the payment and the total cost. A slider tries other amounts, and "Apply Now" opens SBG's real application with the amount filled in.
4. **Answers with sources (RAG).** It answers questions about SBG from a knowledge base built from SBG's public pages, shows numbered citations like [1], and says it doesn't know rather than guessing.
5. **Human handoff.** This kicks in when someone asks for a person, complains, mentions payment trouble, or asks something Sterling can't answer. Sterling creates a ticket with a reference code (like `SBG-7KQ2MX`) and shows the phone number, hours and a callback form. Advisors see the ticket and conversation in an inbox at `/specialist`.
6. **Voice.** You can speak instead of typing, and replies can be read aloud.
7. **Safety.** Social Security, card and bank account numbers are removed before anything goes to the AI or gets saved. It never says "you're approved".
8. **Works without the AI.** With no API key, or when Claude is down, everything still works. Typed questions are answered by quoting the best-matching passage.
9. **"Under the hood" panel.** For each message it shows which rules fired, the search scores, the tool calls, tokens and timing. It's great for a demo.

The one idea to remember: "code decides, the model explains." Who qualifies, and every dollar figure, comes from normal tested code. The AI only understands what people type and explains the results in plain English.

It's an unofficial concept prototype, not affiliated with SBG. Say that up front.

---

2. Tech stack

Frontend

- **Next.js 16 (App Router).** A React framework that gives you web pages and backend API endpoints in one project.
  - *Why:* one codebase and one deploy. API routes can stream a response, and `after()` runs work after the response is sent (used for the advisor summary).

- **React 19.** Builds the UI from components (chat bubbles, cards, panels). *Why:* the standard for interactive apps.
- **TypeScript 5.** JavaScript with types.
  - *Why:* the browser and server share one set of types (`lib/chat/protocol.ts`).
  - So a misspelled event name fails at compile time, not during a demo.
- **CSS Modules + CSS variables.** Each component has its own `.module.css`, and SBG's brand colors are variables in `app/globals.css`.
  - *Why:* SBG's look without a UI library, and styles can't leak between components.
- **next/font** (Host Grotesk, Instrument Sans). These are the fonts sbgfunding.com uses, served from our own server.
- **Web Speech API** (built into the browser). Speech-to-text and text-to-speech.
  - *Why:* no extra service, key or cost, and the server only ever sees text.
- **sessionStorage / localStorage.** Small storage in the browser.
  - *Why:* a page refresh keeps the conversation, and the read-aloud setting is remembered.

AI and search

- **Claude API (`@anthropic-ai/sdk`).** The language model.
  - *Why:* it understands free text, decides which tool to call, and writes answers from the retrieved passages.
  - It is used with streaming (words appear as they're written), tool use, and prompt caching (the fixed instructions are reused, so repeat calls are cheaper and faster).
  - "Effort" is set low so chat replies come back quickly. The model is chosen with `ANTHROPIC_MODEL`.
- **Transformers.js (`@huggingface/transformers`) on ONNX Runtime.** Runs an AI model inside Node.js.
  - *Why:* embeddings are computed locally, so there is no embedding API, no per-call cost, and search queries never leave the server.
- **all-MiniLM-L6-v2.** A small model (about 90 MB) that turns a text into 384 numbers, called an *embedding*. Texts with similar meaning get similar numbers.
  - *Why:* fast on a normal CPU, and accurate enough for 105 passages.
- **BM25,** written by hand in `lib/rag/bm25.ts` (about 75 lines). The classic keyword-ranking formula that search engines use.
  - *Why:* embeddings are fuzzy on exact terms like "7(a)", "UCC" or a phone number, and BM25 catches those.
  - It's small enough that a library wasn't needed.

Validation

- **zod.** Checks the shape of data while the app runs.
  - *Why:* three kinds of data can't be trusted: the state the browser sends, the tool inputs the AI produces, and API request bodies.
  - zod rejects bad data cleanly instead of crashing.

Knowledge-base pipeline

Developer tools, not used while serving users.

- **pdfkit.** Builds the 33-page knowledge-base PDF from 9 Markdown files. *Why:* the brief was "RAG over a PDF", so there is a real PDF.
- **pdfjs-dist.** Reads the PDF back with each line's position and font size.
  - *Why:* to rebuild the headings from the layout, so chunks follow the document's structure.
- **tsx** runs TypeScript scripts directly (`setup` , `ingest` , `eval`). **tar** unpacks the model during `npm run setup` .

Quality and runtime

- **Vitest** runs the 100 tests, and **ESLint** catches bugs and React mistakes.
- **Node.js 20+** runs the server. Its `crypto` module makes random ticket codes, and `fs` writes the ticket file.

Database: none. Login: none. Both are explained in Q6 and Q7.

---

3. Architecture

There are three parts:

1. **The browser.** The chat UI. It *keeps the conversation state* and sends it with every message.
2. **The Next.js server.** Mainly one endpoint, `POST /api/chat` . It runs one "turn" and streams events back, and remembers nothing between messages (it is *stateless*).
3. **Outside pieces.** The Claude API, two JSON files (the search index and the tickets), and offline scripts that build the index.

```
BROWSER  (React UI, holds the state)
   │  POST /api/chat { state, input }
   ▼
API ROUTE  app/api/chat/route.ts
   ▼
ORCHESTRATOR  lib/chat/orchestrator.ts
   ├─ button click → guided flow
   │                  → eligibility + calculator
   └─ typed text → guardrails
        ├─ sensitive → handoff ticket
        └─ normal → Claude + 5 tools
                    (or offline mode)
   │  stream of events (NDJSON)
   ▼
BROWSER  renders text, cards, profile
```

Folder structure

```
app/            pages + API endpoints
  page.tsx        "/" → the chat
  specialist/     "/specialist" → advisor inbox
  api/chat/       run one turn, stream events
  api/handoff/    list / update tickets
  api/health/     model + index status
components/     React UI
  Assistant.tsx   page shell: top bar, layout
  chat/           messages, input, voice,
                  useChat (talks to the API)
  blocks/         cards: question, results,
                  estimate, handoff
  file/           "funding file" side panel
  inspector/      "Under the hood" drawer
  inbox/          advisor inbox
lib/            all the logic (no React)
  chat/           orchestrator + protocol
  agent/          Claude: client, prompt,
                  tools, loop, offline mode
  engine/         profile, flow, eligibility,
                  calculator, guardrails, hours
  rag/            PDF parsing, chunking,
                  embeddings, BM25, search
  handoff/        tickets, summaries, escalate
  sbg/            SBG facts: company,
                  9 products, 18 industries
knowledge/      9 Markdown sources + the PDF
data/           kb-index.json (search index)
scripts/        setup-model, build-kb-pdf,
                ingest, eval
evals/          test questions + results
tests/          100 unit + contract tests
docs/           guides + PDF versions
```

Files to know by name

- `lib/chat/protocol.ts` is the "contract": every request and event shape the browser and server exchange.
- `lib/chat/orchestrator.ts` is the brain of one turn.
- `lib/agent/prompt.ts` holds Sterling's instructions (the system prompt) and builds the per-turn `<app_state>` block.
- `lib/agent/tools.ts` defines the 5 tools, and `lib/agent/agent.ts` is the loop that talks to Claude.
- `lib/engine/*` holds the rules, `lib/rag/retriever.ts` does the search, and `lib/handoff/*` handles tickets.
- `instrumentation.ts` runs once at server start and loads the index and model, so the first question isn't slow.
- `next.config.ts` tells Next.js to load ONNX Runtime as a plain Node package and to ship the index and model files with the build.

Three design decisions worth explaining

1. **Stateless server.** The browser sends the whole state (profile, last 24 messages, and so on) every time.
   - *Pro:* any server copy can answer any message, so it's easy to scale and there's nothing to clean up.
   - *Con:* the browser could send fake state. That's why `sanitizeState()` re-checks every field and caps sizes, and nothing security-critical relies on that state.
2. **Two paths.** Clicks go through a fixed state machine, which is instant, free and predictable. Only typed text goes to the AI.
3. **`lib/` has no React.** The logic can be tested without a browser, and the same calculator code runs on the server and in the browser (the estimate slider).

---

4. Code flow

Flow 0: building the search index

This runs once, before anyone visits.

```
knowledge/source/*.md   (9 docs)
  → npm run kb:pdf      → 33-page PDF
  → npm run ingest
      readPdfLines   lines + font sizes
      buildSections  headings → 73 sections
      chunkSections  ≤150 words → 105 chunks
      embed          384 numbers per chunk
  → data/kb-index.json
```

When the server starts, `instrumentation.ts` calls `warmUp()` so the index and model are already loaded.

Flow A: opening the page

User opens `/` → `app/page.tsx` → `AssistantClient` (browser-only) → `Assistant.tsx` → `useChat()` restores the saved conversation from sessionStorage, or starts empty → the welcome screen shows the starting options.

Flow B: clicking an answer (no AI involved)

Example: the user picks "LLC".

1. **Frontend:** `QuestionCard` → `useChat.send({type:"answer", field:"businessType", value:"llc"})`. Your bubble and an empty Sterling bubble appear immediately.
2. **API:** `POST /api/chat` with `{state, input}`. `route.ts` checks that `input` exists, opens a stream and calls `handleTurn()`.
3. **Backend:**
   - `sanitizeState` → `handleAnswer` → `answerToUpdate` → `mergeProfile` (validated) → sends a `profile` event.
   - Then `advance()` → `nextQuestion()` → sends a `ui` event with the next question card.
4. **Database:** nothing is stored.
5. **Response:** JSON lines stream back: `trace`, `profile`, `ui`, `state`, `done`.
6. **Frontend:** `useChat` reads each line and updates the screen. The funding file shows "LLC" and the progress bar moves.

The whole round trip takes milliseconds and costs nothing in AI.

Flow C: typing a message (Claude available)

Example: *"We're a roofing LLC, 3 years in, about $80K a month. Do you do equipment financing?"*

1. `Composer` → `send({type:"text"})` → `POST /api/chat` → `handleTurn` → `handleText`.
2. **Guardrails.** `checkInput()` removes SSN, card and account numbers, checks the routing rules, and flags injection attempts. Nothing is sensitive here, so it continues.
3. **Request.** `runAgent()` sends the recent text history plus your message, with an `<app_state>` block in front. That block holds the profile, the next question, whether minimums are met, New York time, and whether advisors are in.
4. **Tool calls.** Claude streams back and calls tools:
   - `update_business_profile` with `{industry: roofing, businessType: llc, monthsInBusiness: 36, monthlyRevenue: 80000}`. It is validated, a `profile` event goes out, and four fields light up in the funding file.
   - `search_knowledge_base` with `{query: "equipment financing"}`. `search()` returns the top 5 passages numbered [1] to [5], and the screen shows "Searching SBG's knowledge base…".
5. **Answer.** Claude writes the answer citing [1], streamed as `text` events.
   - The loop ends when Claude answers without asking for a tool, with at most 6 rounds.

- `checkOutput()` appends a correction if it ever said "you're approved".

6. **Wrap-up.** A `sources` event goes out for the cited passages only. Then `advance()` shows the next guided question ("How soon do you need the funds?"), followed by `state` and `done`.

Flow D: typing with no key, or Claude down

`runAgent` fails before writing any text, so the turn goes to `offlineTurn()`:

- If a sensitive number was removed, it sends a fixed safety reply.
- `extractProfile()` uses patterns to find "LLC", "$80K a month", "3 years" and "roofing".
- A plain statement gets "Thanks - I've noted your business type (LLC), ...".
- A question runs `search(text, k=3)`:
  - Low confidence: "I couldn't find that in SBG's materials..." plus a "Talk to a specialist" button.
  - Otherwise: it quotes the best passage's opening sentences and most relevant bullets, with [1].
- If Claude fails after already writing part of a reply, Sterling says "I lost my connection for a moment..." instead.

Flow E: results

1. After the last intake answer, `advance()` → `isIntakeComplete()` → `evaluate(profile)` returns the checks, the ranked products and the estimates.
2. It sends a `results` card, an `estimate` card (only if minimums are met), and quick replies: "Compare my top options", "What documents do I need?", "Talk to a specialist".
3. It saves a fingerprint of the profile (`resultsKey`) so the same results never show twice.
4. In the browser, `EstimateCard` re-runs the calculator itself when you move the slider, with no server call.

Flow F: "Estimate my funding"

It asks only the amount, time in business, credit and revenue → `estimateReady()` → `evaluate()`. If minimums are met you get the estimate card; if not, a results card that explains the gap.

Worked example:

- Points: more than 2 years in business is 11, credit 600–679 is 3, and $50K a month is 14. Total 28, which lands in the 22–30 tier.
- Maximum: $50,000 × 2.25 = $112,500.
- Pricing $50K: × 1.26 = $63,000 repaid over 24 months, twice a month. That's 48 payments of about $1,310, and $13,000 total cost.

Flow G: talking to a human

There are three ways in, all leading to one function:

- the "Talk to a specialist" button → `handleIntent("human")`;
- a typed message that matches a rule ("can I speak to a person", "I can't make this week's payment", "this is a scam"), routed by `checkInput()` with **no AI call**;
- Claude decides (the question isn't in the knowledge base, or it's a complex deal) → the `escalate_to_human` tool.

All three call `escalate()`, which:

1. keeps one ticket per conversation; a second handoff updates the first and never lowers its priority;
2. marks complaint and hardship tickets high priority and leaves out product suggestions;
3. writes an instant template summary and calls `createTicket()`, which saves to `data/handoffs.json`;
4. sends a `handoff` card with the reference code, the phone number, and "Advisors are available now" or "Back Monday at 9am ET" (computed in New York time);
5. after the response is sent, has Claude write a short summary that replaces the template (Next.js `after()`).

The callback form posts to `POST /api/handoff`. It is validated with zod, the message has sensitive numbers removed, and the ticket becomes `callback_requested`.

Flow H: advisor inbox

1. `/specialist` → `Inbox` calls `GET /api/handoff` every 4 seconds.
2. Tickets are sorted with high-priority ones first. Within each group the order is callback requested, new, contacted, closed.
3. The advisor opens one and sees the summary, the callback details, the transcript and the profile.
4. "Mark contacted" or "Close ticket" sends `PATCH /api/handoff`.

Flow I: voice

- **Speaking:** mic → `useSpeechInput` → the browser's SpeechRecognition (Chrome, Edge, Safari) shows the words live. When you pause, the text is sent exactly like typing (Flow C).
- **Read aloud:** when a reply finishes, `speak()` → `speakable()` strips [1], bold and links → the browser reads it, plus the next question. Audio never reaches the server.

Flow J: "Under the hood"

Every step sends `trace` events (time, type, label, data). `useChat` groups them per message, and the `Inspector` drawer lists them. It also calls `GET /api/health` for the model name and index size.

5. Important functions

1. `useChat()`

- **File:** `components/chat/useChat.ts`
- **Does:** the browser side of the chat. It holds the messages and state. `send()` posts to `/api/chat`, reads the stream line by line, updates the UI per event, and saves to sessionStorage. It blocks double-sends while busy.
- **In:** nothing (it's a React hook). `send(input, userText?)` takes a text, answer or intent input.
- **Out:** `{ messages, state, traces, captures, busy, send, reset }`.
- **Called by:** `components/Assistant.tsx`.

2. `handleTurn(request, emit, options)`

- **File:** `lib/chat/orchestrator.ts`
- **Does:** runs one turn end to end.
  - It cleans the state (`sanitizeState`), then routes the input: intent → `handleIntent`, answer → `handleAnswer`, text → `handleText`.
  - It catches errors and shows a friendly message with the phone number.
  - It always ends with `state` and `done`.
- **In:** `{state, input}`, an `emit` callback, and an optional `defer`.
- **Out:** no return value; everything goes out through `emit`.
- **Called by:** `POST` in `app/api/chat/route.ts`.

3. `advance(turn)`

- **File:** `lib/chat/orchestrator.ts`
- **Does:** decides what the screen shows next: the estimate card, the next question, or the final results with quick replies. It uses `profileKey()` so results never repeat.
- **In:** the current turn (state + emit).
- **Out:** sends `ui` events and records `resultsKey` / `estimateKey`.
- **Called by:** `handleIntent`, `handleAnswer`, `handleText`.

4. `checkInput(raw)`

- **File:** `lib/engine/guardrails.ts`
- **Does:** the safety check before any AI call.
  - **Redaction:** `redactPII()` removes SSNs, card numbers (Luhn-checked, so random numbers aren't flagged) and account numbers that appear next to words like "account" or "routing".
  - **Routing:** rules run in order (complaint → hardship → human → existing customer), and the first match wins. Curly apostrophes are normalized first.
  - **Injection:** "ignore previous instructions"-style phrases are flagged (logged, not blocked).

- **In:** the user's text.
- **Out:** `{ text (cleaned), redactions, route, injectionSuspected, matched }`.
- **Called by:** `handleText` and the routing eval. `redactPII` is also used by `POST /api/handoff`, and `checkOutput` runs at the end of `runAgent`.

5. `runAgent(ctx, history, userText)`

- **File:** `lib/agent/agent.ts`
- **Does:** the agent loop. It sends the conversation and tools to Claude, streams text to the browser, runs the tools Claude asks for, sends the results back, and repeats until Claude answers in text (6 rounds at most).
- **Error handling:**
  - An API error returns a failure, so the caller falls back to offline mode.
  - A refusal means tools are never run.
  - A tool call cut off at the token limit is not run.
  - Unreadable tool input is retried up to 2 times.
- **In:** the context (state, emit, trace, sources), the text history and the new message.
- **Out:** `{ ok, text, failure? }`, where `failure` is one of `no_client`, `api_error`, `refusal`, `truncated`.
- **Called by:** `handleText`.

6. `runTool(name, raw, ctx)`

- **File:** `lib/agent/tools.ts`
- **Does:** checks the input with zod, then runs one of the 5 tools: `search_knowledge_base`, `update_business_profile`, `check_eligibility`, `estimate_funding`, `escalate_to_human`. Bad input returns an error message to Claude so it can correct itself, rather than crashing.
- **In:** the tool name, Claude's raw JSON input, and the context.
- **Out:** `{ content (JSON string), isError? }`.
- **Called by:** `runAgent`.

7. `search(query, {k, mode, weights})`

- **File:** `lib/rag/retriever.ts`
- **Does:** hybrid search, in four steps.
  1. Embed the query and compare it with all 105 chunk embeddings (cosine similarity).
  2. Score every chunk with BM25 keywords.
  3. Turn both scores into ranks and fuse them: `1/(60+denseRank) + 0.3/(60+bm25Rank)`, counting only each search's top 20.
  4. Return the top k.

- **Confidence** comes from the best cosine score: 0.5 or more is high, 0.33 or more is medium, anything lower is low.
- **In:** the query. Options: `k` (default 5), `mode` (hybrid / dense / bm25), `weights`.
- **Out:** `{ query, mode, hits[{chunk, dense, bm25, rrf, denseRank, bm25Rank}], confidence, ms }`.
- **Called by:** `runTool` (k=5), `offlineTurn` (k=3), and `scripts/eval.ts`.

8. `evaluate(profile)`

- **File:** `lib/engine/eligibility.ts`
- **Does:** the eligibility engine.
  - It checks the credit, time-in-business and revenue minimums plus the industry restriction. The 4 bank statements are shown as information.
  - It scores all 9 products using fit with the need, SBG's industry recommendation, timeline, amount limits, and product rules (invoices, a bridge repayment event, home equity).
  - It labels each product "strong", "possible" or "not_now", with reasons, blockers and fixes.
  - It adds calculator estimates if the minimums are met.
- **In:** the profile.
- **Out:** `{ checks, meetsMinimums (true / false / null = not known yet), matches, fundingEstimate?, locEstimate?, missing }`.
- **Called by:** `advance`, the `check_eligibility` and `estimate_funding` tools, `escalate`, `buildAppState` (the prompt), `conditionalQuestions` (the flow), and the funding-file panel.

9. `estimateFunding(input)`

- **File:** `lib/engine/estimator.ts`
- **Does:** a copy of SBG's public calculator. Points for time in business (0–11), credit (0–11) and revenue (1–14) add up to a score, and the score picks a tier:

Score	× monthly revenue	Rate factor	Term, payments
10–16	0.75	1.22	12 mo, weekly
17–21	1.5	1.24	15 mo, weekly
22–30	2.25	1.26	24 mo, twice a month
31–36	2.5	1.28	36 mo, monthly

A *rate factor* of 1.26 means you repay $1.26 for every $1 borrowed. A score below 10 gets no automatic estimate.

- **In:** `{ timeInBusiness, creditScore, monthlyRevenue, requestedAmount? }`.
- **Out:** either qualified (with `maxOffer` and an optional `requestedOffer`), or not qualified (with a "home equity line of credit (HELOC) or talk to an advisor" suggestion).

- **Called by:** `evaluate` (server) and `EstimateCard` (browser, for the slider).
- **Next to it:** `estimateLineOfCredit()`. The limit is 2× monthly revenue, rounded up to $5K and capped at $500K.

10. `escalate(ctx, reason, note?)`

- **File:** `lib/handoff/escalate.ts`
- **Does:** the one place a handoff happens: one ticket per conversation, the priority, products only for new-financing leads, an instant summary, the save, the handoff card, and the AI summary afterwards.
- **In:** the context, a reason (`requested`, `complaint`, `hardship`, `existing_customer`, `complex_deal`, `cannot_answer`, `ready_to_apply`) and an optional note.
- **Out:** a `Ticket` (id, status, priority, profile, transcript, summary).
- **Called by:** `handleIntent("human")`, the rule routes in `handleText`, and the `escalate_to_human` tool.

Also worth knowing

- `nextQuestion(profile, intent)` is the state machine. It returns the next unanswered field, plus extra questions about invoices, a repayment event or home equity when they matter.
- `offlineTurn()` is the no-AI mode.
- `chunkSections()` does the chunking.
- `buildAppState()` builds the per-turn `<app_state>` block.

---

6. Resume points

Sterling, AI funding assistant (concept prototype for SBG Funding) · Next.js, TypeScript, Claude API, RAG

- Built a full-stack AI chatbot (Next.js, React, TypeScript, Claude API) that guides small-business owners through SBG Funding's intake, checks them against its published lending minimums, ranks 9 financing products and estimates offers with a port of SBG's public calculator; the guided flow runs as a state machine with no LLM calls.
- Engineered hybrid retrieval (local MiniLM embeddings + BM25, fused with weighted Reciprocal Rank Fusion) over a 105-passage knowledge base; tuned the fusion weight on a 46-question eval set, raising top-3 retrieval accuracy from 89.1% to 93.5% (MRR 0.839) while flagging 100% of off-topic questions as low-confidence.
- Designed safety and reliability layers: PII redaction and rule-based routing of complaints and payment hardship to humans before any model call, warm-handoff tickets with an advisor inbox, a retrieval-only fallback when the LLM is unavailable, and 100 unit and contract tests against a mocked streaming API.

Keep "concept prototype" in the title, and don't add user numbers. Every figure above is one you can explain.

7. Interview questions

Answers are written the way you'd say them out loud.

Basic

Q1. "Explain your project."

Sterling is a website assistant I built as a concept for SBG Funding, a New York small-business lender. Their information is spread across product pages, an FAQ, two calculators and an application. Sterling puts it into one conversation: it asks what an advisor would ask, checks you against SBG's published minimums, matches you to their nine products, estimates an amount with SBG's own calculator formulas, answers questions from SBG's pages with citations, and hands you to a person with full context when that's the right call. The rule I designed around is "code decides, the model explains": eligibility and numbers come from tested code, and Claude only understands what people type and explains the results.

Q2. "Why did you build it? What problem does it solve?"

SBG's site has no guide for the owner. You don't know which product fits, or whether you qualify, until you apply or call. That costs SBG too: advisors talk to people who don't meet the minimums, and customers repeat their story when they're passed along. Sterling pre-qualifies (it never approves), points clear cases to the application, and gives advisors a ticket with the conversation. I built it for this interview because chatbots and AI applications are part of the role.

Q3. "Walk me through a demo."

1. "Find the right financing" brings up the question cards.
2. Type "roofing LLC, 3 years, $80K a month" and four fields light up in the funding file.
3. Finish the questions to get results: a ✓ or ✗ for each minimum, and the best-fit products with reasons.
4. On the estimate card, move the slider and the payment and total cost update.
5. Ask "what documents do I need?" and get a cited answer.
6. "Talk to a specialist" shows a reference code. Open `/specialist` to show the advisor's ticket.
7. Finish by opening "Under the hood" to show the scores and tool calls.

Q4. "How do you know it works?"

Three levels.

- **Automated tests:** 100 of them. They cover the calculator against values worked out by hand from SBG's formulas, eligibility and ranking, the question flow, guardrails, keyword search and offline answers. There are also contract tests that run the AI loop against a fake Claude API.
- **Evaluations:** 46 questions with known correct passages, plus 5 off-topic ones. The right passage is in the top 3 for 93.5% of questions, and all 5 off-topic questions are flagged as low-confidence. All 35 routing test messages route correctly, but I wrote those alongside the rules, so they show the rules do what I intended, not that they generalize.

- **Manual checks:** the production build in a browser at desktop and phone sizes, and a fresh clone set up by following the README.

Q5. "What are its limitations?"

Storage is a JSON file on one server, and the advisor inbox has no sign-in. The knowledge base is a snapshot of SBG's public pages from October 2026. Some ranking weights are my own rules because SBG doesn't publish them, and they're documented. Search misses 3 of the 46 test questions in the top 3. For example, "Do you lend in Texas?": the answer is "all 50 states", but no passage mentions Texas. Voice input doesn't work in Firefox. And the routing rules are tested on cases I wrote, not real conversations.

Technical and code-level

Q6. "How does authentication work?"

There's no login yet. It's a prototype, and I'd say that first. The customer chat doesn't need one, like most website chats, and it never shows account data: existing customers are sent to SBG's portal or their Relationship Manager, because that needs a verified identity.

The real gap is the advisor side. `/specialist` and `/api/handoff` are open, so anyone with the link could read tickets. In production I'd put staff single sign-on in front of both, through SBG's identity provider, and add roles, audit logs and rate limiting on `/api/chat`.

The session id is just a label (the time plus a counter) that groups one conversation, not a secret. What does exist: the server never trusts the browser's state (`sanitizeState` re-checks it), and every API body and AI tool input is validated with zod.

Q7. "Why this database?"

There isn't one, on purpose: the prototype runs with just `npm install`. There are three kinds of data:

- **The search index:** 105 passages × 384 numbers in `data/kb-index.json`, built ahead of time and loaded into memory. A whole search takes about 9 ms, so a vector database would add setup for no gain.
- **Tickets:** in `data/handoffs.json`. Writes run one after another, so two requests can't overwrite each other.
- **The conversation:** in the browser's sessionStorage, sent with each message, so the server stores nothing.

The weak spot is the ticket file: with two servers, each would have its own file. In production, tickets go into SBG's CRM or Postgres, and the knowledge base moves to Postgres with pgvector once it reaches thousands of passages.

Q8. "What is RAG, and how does it work in your project?"

RAG means Retrieval-Augmented Generation. Instead of trusting what the model memorized, you search your own documents and give the model the relevant passages to answer from. There are two halves.

- **Offline, once:** 9 Markdown documents written from SBG's pages become a 33-page PDF.

    - pdfjs parses it, using font sizes to find the headings: 73 sections.
    - Those are split into 105 chunks of at most 150 words that never cross a heading, with a small overlap.
    - Each chunk is embedded with a "Section > Title" label in front, and saved to `kb-index.json`.
- **Live, per question:** Claude calls `search_knowledge_base` with a standalone query. Hybrid search returns the top 5 passages numbered [1] to [5]. Claude must answer only from them and cite them, and the UI shows only the sources it actually cited. If the best match is weak, the tool tells Claude to say it doesn't know and offer a specialist.

Q9. "Why combine keyword search with embeddings? What is RRF?"

Embeddings match meaning: "I can't make payroll" finds "cover payroll gaps". But they're fuzzy on exact terms like "7(a)", "UCC" or a phone number, and BM25 keyword search is the opposite. Their scores are on different scales (cosine runs 0 to 1, BM25 can run past 20), so you can't just add them.

Reciprocal Rank Fusion uses positions instead. A passage gets weight ÷ (60 + its rank) from each search, the two are added, and a passage both searches like rises to the top. On my 46 questions: embeddings alone put the right passage in the top 3 89.1% of the time, keywords alone 71.7%, and combined 93.5%.

Q10. "How do you stop the AI from making things up or promising approval?"

Layers, because no single one is enough.

1. The model never computes eligibility or money. It calls tools that run tested code, and the system prompt forbids inventing figures.
2. Facts come only from retrieved passages, with citations.
3. A confidence gate: if the best match scores under 0.33, the tool tells the model "nothing closely matches, don't answer from general knowledge".
4. An output check catches "you're approved" or "guaranteed" and appends a correction.
5. Sensitive topics never reach the model at all.

I can't make mistakes impossible, but I make them rare and visible: the trace shows exactly which passages it saw.

Q11. "How does tool calling work in your agent?"

I send Claude the conversation plus five tool definitions: a name, a description and a JSON schema for the inputs. Claude replies with text, or with a request like "call search_knowledge_base with {query: "equipment financing"}". My code runs the tool, sends the result back, and asks again. The loop ends when Claude answers in plain text, capped at 6 rounds.

Tool inputs stream as Claude writes them (`eager_input_streaming`), so the "Searching…" status appears right away. In that mode the API doesn't validate the inputs, so I check every one with zod, and if it's invalid I send the error back so Claude can fix it. Within a turn I pass Claude's earlier blocks back unchanged, including its thinking; between turns I keep only the text.

Q12. "How does streaming work? Why not WebSockets?"

`POST /api/chat` returns a stream where each event is one JSON object on its own line. That format is called NDJSON: `{"type":"text","delta":"Sure"}`, `{"type":"ui",…}`, `{"type":"done"}`. The browser splits the stream on newlines and handles each event, so words, cards and profile updates appear as they happen.

WebSockets are for long-lived two-way connections, like games or person-to-person chat. Here each message is one request with a one-way stream back, which works through normal proxies and load balancers. Server-Sent Events would also work, but the browser's `EventSource` can only send GET requests and I need to POST the state. NDJSON over `fetch` was the simplest option.

Q13. "How does the human handoff work?"

There are three triggers: the button, the rules, or Claude's `escalate_to_human` tool. All three go to one function, `escalate()`.

The rules run before the AI: asking for a person, complaint or legal words, and payment-hardship phrases. Those are too important to leave to a model's judgment, and rules are instant, free and predictable.

The ticket has a reference code: "SBG-" plus 6 characters from an alphabet with no look-alikes like 0/O or 1/I, so it can be read over the phone. It also carries the profile, the eligibility result, the last 16 messages with sensitive numbers removed, and a summary.

A template summary is written instantly, and a Claude-written one replaces it after the response is sent, so the customer never waits. There's one ticket per conversation, the priority never drops, and complaint or hardship tickets carry no product pitch.

Scenario-based

Q14. "How would you scale this?"

The design already helps: the server is stateless, so I can run many copies behind a load balancer and any copy can answer any message. Three things need work, in this order:

1. **Tickets:** move from the JSON file to a shared database or the CRM.
2. **Claude calls:** the main cost and the main limit. Prompt caching already cuts the cost of the fixed instructions, and button clicks never call the model, but I'd add per-user rate limits and track cost per conversation.
3. **Embeddings:** they share the web server's CPU. That's fine at this size; at large scale I'd move search to its own service.

Plus monitoring: response time, error rate, and how often it falls back to offline mode.

Q15. "What if 10,000 users come at once?"

Split it by request type. Button clicks are pure code, a few milliseconds and no AI, so a few servers handle them. Each typed message runs a local embedding plus 1 to 6 Claude calls, so Claude's rate limit is the first wall. My plan:

- auto-scale the app servers;
- cap concurrent Claude calls with per-user rate limits (for example in Redis);
- rely on the existing fallback: when Claude returns rate-limit errors, the app answers in retrieval-only mode instead of failing (the SDK retries twice first);
- cache answers to the most common questions;
- move tickets to a real database.

I haven't load-tested it, and I'd say so. I'd do that with a tool like k6 before launch.

Q16. "What if Claude goes down or the key is wrong?"

The app keeps working. With no key, `getClient()` returns nothing and Sterling runs in retrieval-only mode. With a key, any API error is caught: a wrong key (401), a rate limit (429) or an outage (5xx). That message falls back to retrieval-only mode, and the trace says why. If Claude fails after writing part of a reply, Sterling says "I lost my connection for a moment" and offers the phone number. The guided questions, eligibility, calculator and handoffs never needed Claude.

Q17. "SBG changes a product or rule tomorrow. What do you change?"

It depends on whether it's a fact or a rule.

- **A fact** (a new FAQ answer, a fee) lives in the knowledge base. Edit the Markdown in `knowledge/source/`, run `npm run kb:pdf` and `npm run ingest`, then `npm run eval` to check search didn't get worse.
- **A rule** (say the minimum credit score goes from 500 to 550) lives in code. Change it in `lib/sbg/company.ts` (or the product in `lib/sbg/products.ts`), update the tests, and run `npm test`.

One honest catch: the checks read the number from that file, but a few sentences repeat it as plain text: the system prompt, the "See if I qualify" intro, and some labels. Today I'd also search the code for "500+". A small cleanup would build those sentences from the constants.

Tricky follow-ups

These test depth, so answer from understanding. If they ask directly whether you used AI tools, give the straight answer from INTERVIEW_PREP.md: you used an AI coding assistant heavily, and you owned the problem, the decisions and the testing.

Q18. "Why is the BM25 weight 0.3? Why not equal weights?"

I didn't guess it. I measured it with `npm run eval -- --sweep` on the 46 questions:

BM25 weight	Top 1	Top 3	MRR
0 (embeddings only)	73.9%	89.1%	0.819
0.3	**73.9%**	**93.5%**	**0.839**
0.5–0.8	71.7%	93.5%	0.820–0.827
1.0 (equal)	71.7%	89.1%	0.812

Equal weights are actually worse than embeddings alone. Keyword search is noisy on conversational questions, because common words like "cost" match many chunks, so at full weight it pushes good passages down. At 0.3 it only helps with exact terms. For the same reason, I left section names out of the BM25 text. Caveat: 46 questions is a small set, so I'd re-tune on real questions.

MRR (mean reciprocal rank) averages 1 ÷ the position of the right answer, so 1.0 means it's always first.

Q19. "Tell me about a bug you found and how you fixed it."

Payment-hardship routing. "I can't make my payment" must go straight to a person as high priority, never to the AI. In testing, two kinds of messages slipped through:

- "I can't make this week's payment": the pattern allowed only one word between "make" and "payment".
- Messages typed on a phone: phone keyboards insert a curly apostrophe (can’t), and my pattern only had the straight one (').

Both would have reached the AI as ordinary questions. I fixed it by allowing common filler words in between ("this week's", "our", "next", and so on) and converting curly apostrophes to straight ones before any rule runs. I added six test messages so it can't come back. The lesson: write test cases the way real people type.

Bonus story: writing unit tests for eligibility showed SBA 7(a) was marked "worth discussing" for everyone. A general product note ("more paperwork") was counted as a problem with the customer, so product-wide notes are now kept separate from customer-specific cautions.

Q20. "Why is the per-turn info in the user message instead of the system prompt?"

Because of prompt caching. Claude's API can reuse the beginning of a request (here the tool definitions plus the system prompt, about 2,200 tokens), which makes repeat calls cheaper and faster. But that only works if the beginning is byte-for-byte identical. If I put the profile or the current time in the system prompt, it would change on every message and the cache would never be reused.

So the system prompt never changes. Everything that does change (profile, next question, New York time, whether advisors are in) goes in an `<app_state>` block on the newest user message, after the cached part. The inspector shows cache read and write tokens, so you can confirm it's working.

INTERVIEW_PREP.md has more system-design questions.

---

8. Improvements that would make it stand out

You don't need to build these before the interview. They're your answer to "what would you add next?"

1. **A "what Sterling couldn't answer" report** (about half a day).
   - Every low-confidence search and every `cannot_answer` handoff is already recorded.
   - An inbox tab would group those questions, so SBG's team sees which answers to add to the knowledge base, and the eval confirms the fix after re-ingesting.
   - It stands out because the bot improves from real gaps, a feedback loop few projects show.

2. **A bank-statement reader** (1–2 days).
   - SBG decides from four bank statements. Let the user upload one PDF statement, parse it with pdfjs (already in the project), and compute average monthly deposits to fill "monthly revenue".
   - Run it as plain code, never sent to the AI, with account numbers removed by the existing `redactPII`. Keep it in memory only and use sample statements in demos.
   - It stands out because it's specific to SBG's actual process.
3. **Quality checks on every push, plus an AI grader** (half a day for the first part).
   - A GitHub Action runs `npm test` and `npm run eval` and fails the build if top-3 accuracy drops below 90%.
   - Then add an "LLM-as-judge" check: a second Claude call verifies that every sentence in 30 sample answers is supported by a cited passage.
   - It shows you measure AI quality rather than eyeballing it.
4. **Spanish mode** (about a day).
   - Many US small-business owners prefer Spanish. Browser speech supports `es-US` and Claude can answer in Spanish.
   - The real problem: MiniLM is English-only, so you'd switch to a multilingual embedding model and re-run the eval in both languages.
   - It's real business value and a real RAG problem to talk about.
5. **Production basics** (1–2 days): staff sign-in on the inbox, tickets in Postgres or a CRM webhook, rate limiting on `/api/chat`, and a Slack alert for high-priority tickets. This turns Q6, Q7 and Q14 into things you've actually built.

Not verified yet

Don't guess about these in an interview.

- **Live Claude behavior.** The app has never run with a real API key. The AI path is tested against a fake API that sends real-format events, so how Claude phrases answers and how often it calls each tool is unverified. Before the interview:
  1. Put your key in `.env.local`.
  2. Run `npm run build && npm start`.
  3. Send a few messages and open "Under the hood".
  4. From your second typed message on, `cacheReadTokens` should be above zero. The cached part is about 2,200 tokens, over the default model's 512-token minimum. Some older models need up to 4,096, and then caching silently doesn't happen.
- **The AI-written handoff summary.** It only runs with a key and has never been tested live.
- **Voice.** It was never tested with a real microphone, because the test browser had none. Try it in Chrome.

- **Your account's access.** The default model or the fallback beta might not be enabled for your key. `ANTHROPIC_MODEL` and `ANTHROPIC_FALLBACKS=off` fix that (see the README troubleshooting table).
- **Dev-server reload loop.** About 1 in 6 dev restarts reloads the page repeatedly, and the cause is unknown. Demo with the production build.
- **Rules written for the prototype, not by SBG.** Some ranking numbers are invented: how well each product fits each need, +1 for SBG's industry pick, −1.5 when the timeline is too tight, and the HELOC taking about 21 days. They're listed in RESEARCH_NOTES.md; say so if asked.
- **Third-party review information** is secondhand, because some review sites blocked direct access.